



دانشگاه اصفهان

درس پردازش سیستم‌های داده حجیم

پروژه پایانی

استاد: دکتر محمدعلی نعمت بخش

طراحان: علی معینیان - مریم صفوی - بهنوش بهیانی

Real-Time Fraud Detection Lakehouse Pipeline

سازمان‌های مالی، امروزه با چالشی دوگانه روبرو هستند: از یک طرف، حجم عظیمی از تراکنش‌های مالی به صورت real-time تولید می‌شود و از سوی دیگر، تشخیص تقلب باید در کسری از ثانیه رخ دهد. معماری‌های سنتی که داده را ابتدا در یک Data Lake ذخیره می‌کنند و سپس به یک Data Warehouse منتقل می‌کنند، با مشکلاتی از قبیل High Cost of Ownership و Data Staleness، Reliability مواجه هستند.

🔥 در ابتدا، سه مشکل بیان شده را جستجو کنید و تعریف هر کدام را بنویسید.

در این پروژه، شما باید یک Pipeline کامل طراحی و پیاده‌سازی کنید که ایده‌های مطرح شده در مقالات علمی معتبر را در یک سیستم واقعی پیاده‌سازی کنید. قبل از شروع هر فاز، مطالعه مقالات مربوط به آن اجباری است.

مقالات اجباری

نام مقاله	لینک دانلود	چرا باید بخوانید؟
Lakehouse: A New Generation of Open platforms	لینک	درک معماری Lakehouse
Structured streaming	لینک	درک مفاهیم پایه
Photon	لینک	درک vectorized execution
Fraud Detection	لینک	Baseline مقایسه و معماری پایه

دیتاست پروژه

دیتاست Credit Card Fraud Detection را می‌توانید از طریق [لینک](#) دانلود کنید. دیتاست نام‌برده از معروف‌ترین دیتاست‌های موجود در سایت [Kaggle](#) برای وظیفه Fraud Detection است. مطالعه Notebook‌های موجود می‌تواند به شما در حل بهتر این تمرین کمک کند.

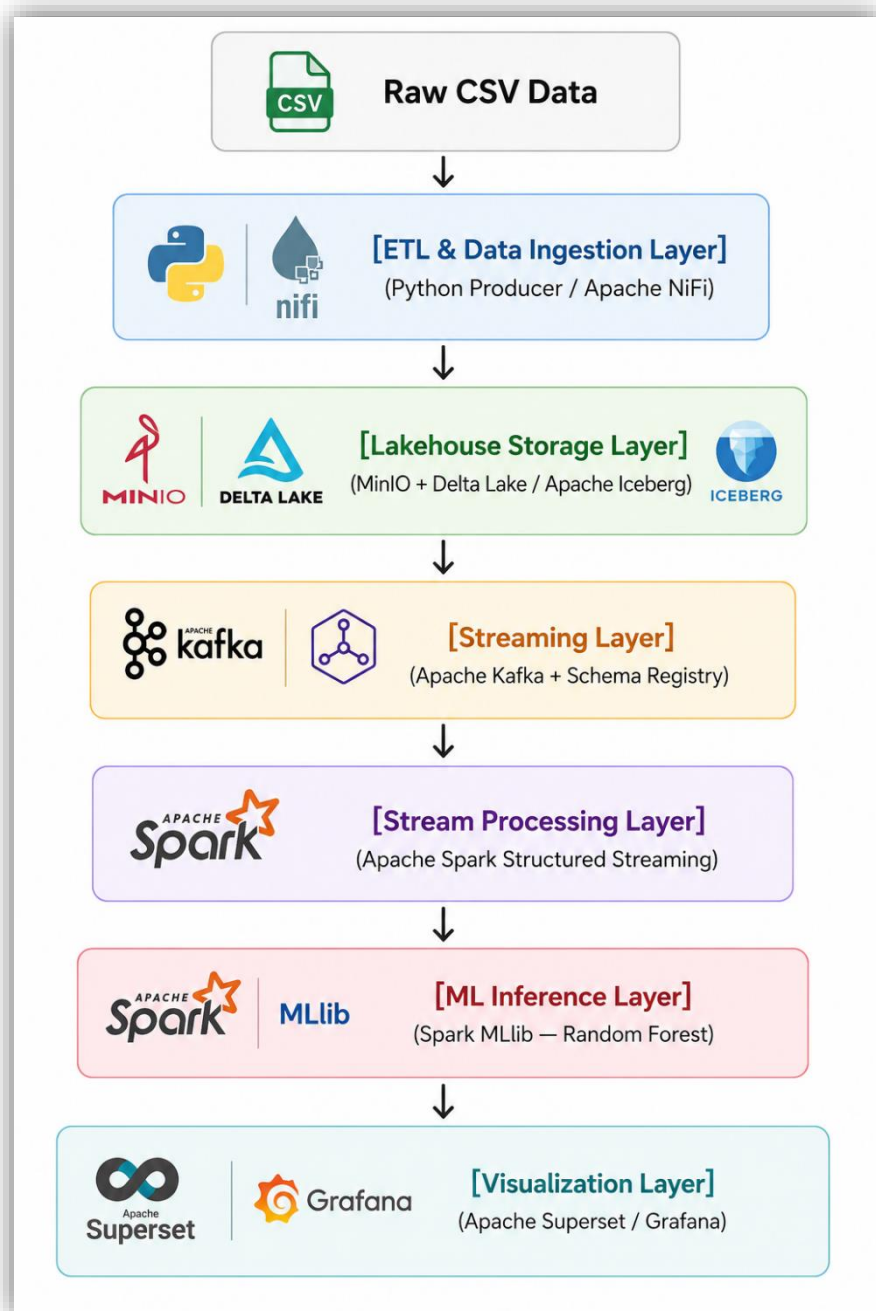
حتما به این مورد دقت داشته باشید که در مقاله Baseline چه تقسیم‌بندی‌هایی در خصوص دیتاست انجام پذیرفته است؟

🔥 دیتاست داده شده را به صورت کامل بررسی کنید. در داکيومنت خود اشاره کنید که Feature‌های این دیتاست چه هستند و چه معنایی دارند؟

🔥 با توجه به آنچه آموخته‌اید، دیتاست را از ابعاد مختلف بررسی کنید (Data Preprocessing).

یادتان باشد که بدون دلیل حق حذف هیچ داده‌ای را ندارد! اگر گفتید چرا؟

معماری سیستم



پیشنهاد ما این است که تمامی اجزا را با Docker و Docker Compose راه اندازی کنید. یک فایل docker-compose.yml بسازید که تمام سرویس‌ها را با یک دستور بالا بیاورد. البته باید به این نکته دقت کنید که ورژن‌های ابزارهای مورد نیاز حتما باید با هم، همخوانی داشته باشد و این، از چالش‌های پیش‌رو خواهد بود.

مطالعه و مستندسازی نظری

پس از مطالعه مقالات، باید در گزارش نهایی خود به سوالات زیر پاسخ داده باشید:

🔴 بر اساس مقاله Lakehouse، معماری Two-tier چه مشکلاتی دارد؟ چرا Lakehouse این مشکلات را حل می‌کند؟ در پیاده‌سازی خود چگونه از مفهوم Lakehouse استفاده کرده‌اید؟

🔴 بر اساس مقاله Structured Streaming، تفاوت Micro batch Execution و Continuous Processing چیست؟ Watermark چه مشکلی را حل کرده است و در سیستم شما چه نقشی داشته است؟ منظور از exactly-once semantics چیست؟

🔴 بر اساس مقاله Photon، چرا Columnar Storage نسبت به row-based storage در Analytical Workload ها برتری دارد؟ Apache Parquet چگونه این مزیت را پیاده می‌کند؟

🔴 بر اساس مقاله Fraud Detection، معماری پیاده سازی شده را توضیح دهید. متریک‌های اندازه‌گیری کدامند؟ تمام آن‌ها را توضیح دهید.

🔴 **چالش‌های سیستم‌های توزیع شده:** در صورت نیاز به Scale کردن سیستم، تنظیمات پارتیشن‌های Kafka و تعداد Executors های Spark چگونه باید هماهنگ شوند تا Bottleneck ایجاد نشود؟ همچنین در پردازش‌های مبتنی بر پنجره زمانی (Window)، Spark چگونه از State Store استفاده می‌کند و چالش پر شدن حافظه (OOM) چگونه مدیریت می‌شود؟

مرحله اول: راه اندازی محیط و Lakehouse Layer

> مرتبط با مقاله Lakehouse <

در این فاز باید لایه ذخیره‌سازی Lakehouse را راه‌اندازی نمایید.

طبیعتاً، فایل docker-compose.yml شما باید حداقل سرویس‌های زیر را داشته باشد:

```
services:
  - minio                # object store (S3-compatible storage for Delta Lake files)
  - zookeeper            # coordination service required by Kafka
  - kafka                # message broker for real-time transaction streaming
  - schema-registry      # Confluent Schema Registry for Kafka message schema enforcement
  - spark-master          # Spark cluster master node
  - spark-worker          # Spark cluster worker node (can scale horizontally)
  - superset             # visualization dashboard (or grafana as alternative)

# Optional (bonus):
- mlflow                 # experiment tracking and model registry
```

🔴 یک MinIO bucket به نام fraud-lakehouse بسازید. دیتاست را با فرمت Apache Parquet در MinIO ذخیره کنید و Delta Lake را روی این bucket فعال نمایید.

🔥 در این گام شما باید ACID Transaction را نشان دهید. یک batch write را در حین اجرا قطع کنید. نشان دهید که Delta Lake به صورت خودکار Rollback انجام می‌دهد. در نهایت، درباره ویژگی Time Travel مطالعه کنید، آن را تعریف کنید و به صورت عملی هم نشان دهید.

```
# Delta Lake Time Travel example
df_old = spark.read.format("delta") \
    .option("versionAsOf", 0) \
    .load("s3a://fraud-lakehouse/transactions")
```

🔥 در گزارش خود کامل شرح دهید که قابلیت Time Travel چرا در Fraud Detection مهم است؟ اگر ACID نبود، چه اتفاقی می‌افتاد؟

🔥 نشان دهید اگر یک رکورد با Schema اشتباه (مثلا ستون اضافه یا تایپ نادرست) وارد شود، Delta Lake آن را نخواهد پذیرفت.

🔥 **بهینه‌سازی Lakehouse:** برای بهبود سرعت کوئری‌ها در تحلیلات آتی، باید از دستور OPTIMIZE و Z-ORDER روی یکی از ستون‌های پرکاربرد استفاده کنید و تاثیر آن را شرح دهید. همچنین نشان دهید چگونه با استفاده از دستور VACUUM، فایل‌ها و نسخه‌های قدیمی را حذف می‌کنید تا فضای ذخیره‌سازی هدر نرود.

🔥 **مدیریت تکامل اسکیم:** نشان دهید اگر در حین اجرای Pipeline استریمینگ، یک فیلد جدید به JSON‌های ورودی در Kafka اضافه شود، Delta Lake چگونه می‌تواند با استفاده از ویژگی mergeSchema بدون متوقف شدن سیستم، این فیلد جدید را به جداول خود اضافه کند.

مرحله دوم: Data Ingestion

< مرتبط با مقاله [Structured Streaming](#)

یک Kafka Producer بنویسید که دیتاست را بخواند، هر رکورد را به صورت JSON در Topic ساخته شده ارسال کند. نرخ ارسال باید قابل تنظیم باشد (TPS). منطبق با مقاله مربوطه، باید از TPS10 و TPS40 پشتیبانی کند.

هنگام ارسال هر رکورد به Kafka، یک فیلد event_timestamp با زمان واقعی ارسال (Wall-Clock time) به JSON اضافه کنید. این فیلد در مراحل بعدی برای پیاده‌سازی Watermark و Window Aggregation الزامی است. زیرا ستون Time در دیتاست اصلی یک Timestamp واقعی نیست و تنها تعداد ثانیه‌های سپری شده از اولین تراکنش را نشان می‌دهد.

نمونه JSON هر پیام ارسالی به Kafka:

```
{
  "event_timestamp": "2025-01-15T10:23:45.123456",
  "Time": 406,
  "V1": -1.35,
  "Amount": 149.62,
  "Class": 0
}
```

🔥 از Confluent Schema Registry استفاده کنید تا Schema پیام‌های Kafka ثبت شود. این مفهوم مستقیماً با Schema Enforcement مطرح شده در مقاله Lakehouse مرتبط است.

مرحله سوم: Stream Processing

< مرتبط با مقاله Structured Streaming >

یک Spark Structured Streaming Job بنویسید که:

- از Kafka Topic مربوطه بخواند
- پیام‌های JSON را Parse کند
- نتایج را به MinIO/Delta Lake بنویسد.

🔥 باید Watermark را پیاده سازی کنید:

```
df_with_watermark = df \
    .withWatermark("event_timestamp", "10 seconds") \
    .groupBy(
        window(col("event_timestamp"), "1 minute", "30 seconds")
    )
```

در گزارش خود توضیح دهید که Watermark چه مشکلی را حل می‌کند؟ اگر Watermark نبود، در یک Stream طولانی چه اتفاقاتی می‌افتاد؟ و عدد 10 Seconds چرا انتخاب شده است؟

برای Watermark از فیلد event_timestamp که در مرحله Kafka Producer اضافه کردید استفاده کنید. نه از ستون Time دیتاست اصلی

🔥 نشان دهید که سیستم شما در صورت خرابی و شروع مجدد، داده ای را دوباره پردازش نمی‌کند. این کار از طریق پیاده سازی Checkpoint در Spark انجام می‌شود.

```
query = df_processed \
    .writeStream \
    .format("delta") \
    .option("checkpointLocation", "s3a://fraud-lakehouse/checkpoints/") \
    .outputMode("append") \
    .start()
```

🧨 منطبق با مقاله Spark Structured Streaming هر سه حالت موجود در خروجی (outputMode) را توضیح دهید و تحلیل کنید چرا در کد قبلی، از append استفاده شده است؟ اصلاً آیا انتخاب درستی است یا باید تغییر کند؟

🧨 **مهندسی ویژگی بلادرنگ:** یک Aggregation مبتنی بر پنجره زمانی (Window) به پایپ‌لاین استریمینگ اضافه کنید (مثلاً محاسبه تعداد کل تراکنش‌ها در ۱ دقیقه گذشته یا مجموع مبالغ تراکنش‌ها در یک پنجره زمانی مشخص) و این ویژگی جدید را همراه با رکورد اصلی برای استفاده در مدل آماده کنید.

مرحله چهارم: ML Training

< مرتبط با مقاله Fraud Detection >

🧨 مدل را با استفاده از Spark MLlib آموزش دهید:

```
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Feature columns
feature_cols = [f"V{i}" for i in range(1, 29)] + ["Time", "Amount"]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

rf = RandomForestClassifier(
    labelCol="Class",
    featuresCol="features",
    numTrees=100,
    seed=42
)
```

🧨 دیتاست‌های که برای وظیفه Fraud Detection استفاده می‌شوند، بسیار Imbalanced هستند. دلیل آن را بیان کنید.

🔥 برای رفع ایراد Imbalance بودن دیتاست، به انتخاب خود، روشی را پیاده سازی کنید. دلیل انتخاب روش خود را باید توضیح دهید.

🔥 مدل آموزش دیده را در Spark Structured Stream بارگذاری کنید و روی هر micro-batch از Kafka اعمال کنید.

```
# Load model
from pyspark.ml import PipelineModel
model = PipelineModel.load("s3a://fraud-lakehouse/models/rf_model")

# Apply to stream
predictions = model.transform(streaming_df)
```

🔥 در نهایت، برای هر تراکنشی که به عنوان Fraud شناسایی می شود، یک پیام هشدار به یک Kafka Topic جداگانه به نام fraud-alerts ارسال کنید.

🔥 **(امتیاز مثبت) ادغام با MLflow:** ابزار MLflow را به صورت یک سرویس در Docker Compose اضافه کنید. تمامی آزمایش های مربوط به آموزش مدل (شامل هایپرپارامترها و متریک هایی مانند $F1 - Score$ و $ROC - AUC$) باید در MLflow ثبت (Log) شوند. همچنین مدل نهایی باید در Model Registry این ابزار ثبت شده و Spark Streaming مدل را مستقیماً از MLflow برای استنتاج بلادرنگ بارگذاری کند.

مرحله پنجم: ارزیابی

< مرتبط با مقاله Fraud Detection >

🔥 ابتدا باید Latency را اندازه گیری کنید. به مقاله نام برده مراجعه کنید تا تعریف Latency را پیدا کنید و برای ۲ سناریو TPS10 و TPS40 پیاده سازی کنید.

به این نکته حتما دقت کنید که اندازه گیری Latency در سناریوهای TPS10 و TPS40 تنها بر روی چه تقسیم بندی ای از دیتاست انجام شده است؟! حتما به مقاله Baseline خود مراجعه کنید.

نتایج خود را در جدول زیر یادداشت کنید:

Metric	Base Paper	Your Pipeline	Difference	Reason?
Avg Latency @ 10 TPS	0.8s	?	?	?
Avg Latency @ 40 TPS	0.9s	?	?	?

🔥 اندازه گیری دقت مدل:

Metric	Base Paper	Your Pipeline
Precision	0.34	?
Recall	0.88	?
F1-Score	0.49	?
ROC-AUC	0.94	?

🔥 اگر نتایج شما با مقاله تفاوت دارد باید علت آن را توضیح دهید. عوامل زیر را بررسی کنید:

- تفاوت در سخت افزار
- تفاوت در تنظیمات Spark
- تفاوت در Preprocessing
- تفاوت در مواجهه با Class Imbalance

مرحله ششم: Visualization

< مرتبط با مقاله Lakehouse >

با استفاده از Apache Superset یا Grafana، یک داشبورد بسازید که موارد زیر را نشان دهد:

- تعداد تراکنش‌های عادی در مقایسه با تقلبی در هر دقیقه
- مجموع مبلغ تراکنش‌های مشکوک
- Latency در طول زمان
- نرخ Fraud در Sliding Window

به یاد داشته باشید که Superset اولویت بیشتری برای انتخاب دارد. در صورت استفاده از Grafana باید یک لایه میانی مانند PostgreSQL به Pipeline اضافه کنید که داده‌های پردازش شده را از Spark دریافت کند و این پیچیدگی اضافهای به پروژه تحمیل می‌کند.

نشان دهید می‌توانید مستقیماً از Delta Lake روی MinIO با استفاده از Spark SQL، کوئری بزنید. این مفهوم Direct Access است که در مقاله Lakehouse روی آن تاکید دارد.

```
SELECT
  window_start,
  COUNT(*) as total_transactions,
  SUM(CASE WHEN prediction = 1 THEN 1 ELSE 0 END) as fraud_count,
  AVG(Amount) as avg_amount
FROM fraud_lakehouse.transactions
GROUP BY window_start
ORDER BY window_start DESC
LIMIT 10;
```

گزارش نهایی

گزارش نهایی شما باید شامل بخش‌های زیر باشد:

۱: مقدمه و انگیزه 📌 چرا این مسئله مهم است؟

۲: مرور ادبیات 📌 خلاصه هر ۴ مقاله و ارتباط آن‌ها با قسمت‌های مختلف پروژه شما

۳: معماری سیستم 📌 دیاگرام کامل Pipeline را با توضیح جزئیات هر Component آن شرح دهید

۴: نتایج به دست آمده و مقایسه با baseline 📌 جدول‌های نامبرده را تکمیل کنید

۵: تحلیل 📁 اگر پاسخ‌های شما با آنچه در مقاله پایه آمده است اختلاف دارد، علت آن را کشف کنید

۶: چالش‌ها 📁 تمامی چالش‌هایی که در طول پیاده‌سازی با آن مواجه شدید را به همراه راه حل، بیان کنید

۷: نتیجه‌گیری 📁 چه کردید؟ چه یاد گرفتید و چه پیشنهادهایی برای بهبود دارید؟

۸: منابع 📁 تمام منابع مورد استفاده خود را لیست کنید (مقالات، سایت‌ها، داکيومنت‌ها و AI)

نکات بسیار مهم

📌 پروژه شما ارائه آنلاین خواهد داشت و قسمت مهمی از نمره نهایی شما به آن بستگی دارد. در ارائه آنلاین، علاوه بر توضیح کامل Pipeline پیاده‌سازی شده، باید بر مقالات نام‌برده تسلط داشته باشید و پاسخگو سوالات TAها باشید.

📌 استفاده از AI هیچ ایرادی ندارد اما تسلط شما بر تمام جزئیات Pipeline و مفاهیم بسیار اهمیت دارد.